

BSCCS2005 : OPE1 Revision Questions with Test Cases and Solutions
--

Overview

A student will be presented with 7 questions out of which 5 are graded and 2 are optional (challenging). Out of the 5 graded questions, he/she should answer 4 questions correctly, passing all the private test cases to get a full score. In case they attempt all 5, only the best 4 scores will be considered. We configure 2 copies of each type on the portal, and using a randomization script, every student sees one copy of each type on the exam portal.

The challenging questions are not graded. The purpose of these questions is to engage students who finish the other questions early.

0.1 Copy Constructor

Problem Statement

In a games event, Player p1 participates a set of games. Player p2 also participates all the games participated by p1 except the second game, in place of which p2 participates another game. Write a program that defines two classes Player and Event. Define copy constructor to create p2 from p1 such that changing the values of instance variables of either p2 or p1 does not affect the other one. The code takes name of player p2 and the new game participating by p2 as input.

- Class Player has/should have the following members.
 - Private instance variables String name and String[] games to store name and games participated respectively
 - Define required constructor(s)
 - Accessor methods getName() and getGames(int) to get the name of the player and the game at a specific index respectively.
 - Mutator methods setName(String) and setGames(int,String) to set the name of the player and the game at a specific index respectively.
- Class Event has method main that does the following.
 - Two objects of Player p1 and p2 are created. p2 is created using p1
 - name of p2 and second game participated by p2 are updated by taking the input
 - Finally, name of p1, p2 and second game participated by p1 and p2 are printed

What you have to do

- Define constructor(s) in class Player.

Template Code

```
import java.util.*;
class Player{
    String name;
    String[] games;
    /*******Define constructor(s) here
    public void setName(String n) {
        name = n;
    }
    public void setGames(int indx, String g) {
        games[indx] = g;
    }
    public String getName() {
```

```

        return name;
    }
    public String getGames(int indx) {
        return games[indx];
    }
}
public class Event{
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String[] games = {"Throwball","Javelin","Volleyball"};
        Player p1 = new Player("Ranjit", games);
        Player p2 = new Player(p1);
        p2.setName(sc.next());
        p2.setGames(1,sc.next());
        System.out.println(p1.getName() + ": " + p1.getGames(1));
        System.out.println(p2.getName() + ": " + p2.getGames(1));
    }
}

```

Public test case 1:

Input:

Sunil Football

Output:

Ranjit: Javelin

Sunil: Football

Public test case 2:

Input:

Pai Cricket

Output:

Ranjit: Javelin

Pai: Cricket

Private test case 1:

Input:

Kapil Cricket

Output:

Ranjit: Javelin
Kapil: Cricket

Solution:

```
import java.util.*;
class Player{
    String name;
    String[] games;
    public Player(String n, String[] g) {
        name = n;
        games=g;;
    }
    public Player(Player p) {
        this.name = p.name;
        this.games = new String[p.games.length];
        for(int i = 0; i < games.length; i++) {
            this.games[i] = p.games[i];
        }
    }
    public void setName(String n) {
        name = n;
    }
    public void setGames(int indx, String g) {
        games[indx] = g;
    }
    public String getName() {
        return name;
    }
    public String getGames(int indx) {
        return games[indx];
    }
}
public class Event{
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String[] games = {"Throwball","Javelin","Volleyball"};
        Player p1 = new Player("Ranjit", games);
        Player p2 = new Player(p1);
        p2.setName(sc.next());
        p2.setGames(1,sc.next());
        System.out.println(p1.getName() + ": " + p1.getGames(1));
    }
}
```

```
        System.out.println(p2.getName() + ": " + p2.getGames(1));  
    }  
}
```

0.2 Classes

Problem Statement

Write a java program that, given input as name, salary to create an array of Employee objects, prints the bonus of employees. Complete the program as specified below:

Abstract class **Employee** has the following members:

- Private instance variables String **name** and double **salary**
- Constructor to initialize the instance variables
- Accessor method for **salary**, **name**
- Abstract method **public abstract void printBonus()**

Classes **Manager** and **Director** should be defined in such a way that any object of **Manager** or **Director** can be assigned to a reference variable of type **Employee**.

- Class **Manager** should have constructor to initialize the instance variables. For an object of **Manager**, method **printBonus** should print the name of employee followed by his/her bonus i.e **getSalary() * 0.1**, where **getSalary()** is a Accessor method of class **Employee**
- Class **Director** should have constructor to initialize the instance variables. For an object of **Director**, method **printBonus** should print the name of employee followed by his/her bonus i.e **getSalary() * 0.15**, where **getSalary()** is a Accessor method of class **Employee**

Class **Organisation** has the following members:

- Method **main** accepts the **name**, **salary** of a **Manager** object followed by that of a **Director** object and stores the objects in an **Employee** array, and then invokes the method **printBonus** for each object in the array

What you have to do

- Define subclasses **Manager** and **Director**

[Abstract Class:Sonam]

Test Cases

Public test case 1 (Input):

Ashok 30000.00

Swaraj 40000.00

Output:

Ashok : 3000.0
Swaraj : 6000.0

Public test case 2 (Input):

Srinivas 50000.00
Sureka 453200.00

Output:

Srinivas : 5000.0
Sureka : 67980.0

Private test case 1 (Input):

Rahul 40000.00
Usha 234000.00

Output:

Rahul : 4000.0
Usha : 35100.0

Private test case 2 (Input):

Saurab 56000.00
Harsha 23000.00

Output:

Saurab : 5600.0
Harsha : 3450.0

Template Code

```
import java.util.*;
abstract class Employee {
    private String name;
    private double salary;

    public Employee(String n, double s) {
        name = n;
        salary = s;
    }

    public double getSalary() {
        return salary;
    }
}
```

```

        public String getName() {
            return name;
        }

        public abstract void printBonus();
    }

    //***** DEFINE class Manager

    //***** DEFINE class Director

    public class Organisation {

        public static void main(String[] args) {
            Scanner sc = new Scanner(System.in);
            Employee[] eArr = new Employee[2];
            eArr[0] = new Manager(sc.next(), sc.nextDouble());
            eArr[1] = new Director(sc.next(), sc.nextDouble());
            eArr[0].printBonus();
            eArr[1].printBonus();
            sc.close();
        }
    }
}

```

Solution:

```

import java.util.*;
abstract class Employee {
    private String name;
    private double salary;

    public Employee(String n, double s) {
        name = n;
        salary = s;
    }

    public double getSalary() {
        return salary;
    }

    public String getName() {
        return name;
    }
}

```

```

    }

    public abstract void printBonus();
}

public class Manager extends Employee {
    public Manager(String name, double salary) {
        super(name, salary);
    }

    public void printBonus() {
        System.out.println(getName() + " : " + getSalary() * 0.1);
    }
}

public class Director extends Employee {
    public Director(String name, double salary) {
        super(name, salary);
    }

    public void printBonus() {
        System.out.println(getName() + " : " + getSalary() * 0.15);
    }
}

public class Organisation {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Employee[] eArr = new Employee[2];
        eArr[0] = new Manager(sc.nextLine(), sc.nextDouble());
        eArr[1] = new Director(sc.nextLine(), sc.nextDouble());
        eArr[0].printBonus();
        eArr[1].printBonus();
        sc.close();
    }
}

```

0.3

Abstract classes and Interfaces

Problem Statement

Write a program that takes input to create an array of **Staff** objects and prints the remuneration of each staff for their duties. **Faculty** should be assigned to **Invigilation** duty, and **Hod** should be assigned to **Squad** duty.

Abstract class **Staff** has the following members:

- String **name**, String **duty** and int **num** as private instance variables, where **duty** is the type of duty which may be either **Invigilation** or **Squad**, and **num** is the number of hours of duty
- constructor to initialize the instance variables
- required accessor methods
- method **toString()** to return the name of the staff
- abstract method **abstract String remuneration()**

Class **Faculty** is a child class of **Staff** and has the following members:

- constructor to initialize the instance variables
- method **remuneration()** returns the remuneration as **num*150**, if the **duty** is **Invigilation**, otherwise returns the string **Wrong duty assigned**

Class **Hod** is a child class of **Staff** and has the following members:

- constructor to initialize the instance variables
- method **remuneration()** returns the remuneration as **num*500**, if the **duty** is **Squad**, otherwise returns the string **Wrong duty assigned**

Class **ExamBranch** has the following members:

- method **printStaffRemunerations** that prints the name and remuneration of the staff if they are assigned valid **duty**, otherwise prints the name and the corresponding error message for each staff
- **main** method accepts the name, duty and number of hours of two **Faculty** members followed by that of two **Hods**, and invokes method **printStaffRemunerations**

What you have to do

- Define method **printStaffRemunerations** in class **ExamBranch**

Test Cases

Public test case 1 (Input):

Vishwanath Squad 3
Dileep Invigilation 4
Kavitha Invigilation 2
Baburao Squad 4

Output:

Vishwanath : Wrong duty assigned
Dileep : 600
Kavitha : Wrong duty assigned
Baburao : 2000

Test Cases

Public test case 2 (Input):

Koteshwar Invigilation 3
Koushik Invigilation 4
Kalavathi Squad 2
Yugandar Squad 4

Output:

Koteshwar : 450
Koushik : 600
Kalavathi : 1000
Yugandar : 2000

Private test case 1 (Input):

Somayajulu Invigilation 1
Sobhanbabu Squad 4
Laxmi Squad 3
Divya Squad 3

Output:

Somayajulu : 150
Sobhanbabu : Wrong duty assigned
Laxmi : 1500
Divya : 1500

Private test case 2 (Input):

Shaurya Squad 1
Karthik Squad 4
Deepa Invigilation 3
Nandini Invigilation 3

Output:

Shaurya : Wrong duty assigned
Karthik : Wrong duty assigned
Deepa : Wrong duty assigned
Nandini : Wrong duty assigned

Template Code

```
import java.util.*;
abstract class Staff {
    private String name;
    private String duty;
    private int num;
    public Staff(String name, String duty, int num) {
        this.name = name;
        this.duty = duty;
        this.num = num;
    }
    public String getDuty() {
        return duty;
    }
    public int getNum() {
        return num;
    }
    public String toString() {
        return name;
    }
    abstract String remuneration();
}
class Faculty extends Staff{
    public Faculty(String name, String duty, int num) {
        super(name, duty, num);
    }
    public String remuneration() {
        if(getDuty().equals("Invigilation"))
            return getNum()*150+"";
        else
            return "Wrong duty assigned";
    }
}
```

```

}
class Hod extends Staff{
    public Hod(String name, String duty, int num) {
        super(name, duty, num);
    }
    public String remuneration() {
        if(getDuty().equals("Squad"))
            return getNum()*500+"";
        else
            return "Wrong duty assigned";
    }
}
public class ExamBranch {

    //***** Define method printStaffRemunerations here

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Staff[] sArr = new Staff[4];
        sArr[0] = new Faculty(sc.next(), sc.next(), sc.nextInt());
        sArr[1] = new Faculty(sc.next(), sc.next(), sc.nextInt());
        sArr[2] = new Hod(sc.next(), sc.next(), sc.nextInt());
        sArr[3] = new Hod(sc.next(), sc.next(), sc.nextInt());
        printStaffRemunerations(sArr);
    }
}

```

Solution:

```

import java.util.*;
abstract class Staff {
    private String name;
    private String duty;
    private int num;
    public Staff(String name, String duty, int num) {
        this.name = name;
        this.duty = duty;
        this.num = num;
    }
    public String getDuty() {
        return duty;
    }
}

```

```

        public int getNum() {
            return num;
        }
        public String toString() {
            return name;
        }
        abstract String remuneration();
    }
    class Faculty extends Staff{
        public Faculty(String name, String duty, int num) {
            super(name, duty, num);
        }
        public String remuneration() {
            if(getDuty().equals("Invigilation"))
                return getNum()*150+"";
            else
                return "Wrong duty assigned";
        }
    }
    class Hod extends Staff{
        public Hod(String name, String duty, int num) {
            super(name, duty, num);
        }
        public String remuneration() {
            if(getDuty().equals("Squad"))
                return getNum()*500+"";
            else
                return "Wrong duty assigned";
        }
    }
    public class ExamBranch {
        public static void printStaffRemunerations(Staff[] obj) {
            for (int i = 0; i < obj.length; i++) {
                System.out.print(obj[i]+" : ");
                System.out.println(obj[i].remuneration());
            }
        }
        public static void main(String[] args) {
            Scanner sc = new Scanner(System.in);
            Staff[] sArr = new Staff[4];
            sArr[0] = new Faculty(sc.next(), sc.next(), sc.nextInt());
            sArr[1] = new Faculty(sc.next(), sc.next(), sc.nextInt());

```

```
        sArr[2] = new Hod(sc.next(), sc.next(), sc.nextInt());  
        sArr[3] = new Hod(sc.next(), sc.next(), sc.nextInt());  
        printStaffRemunerations(sArr);  
    }  
}
```

1 Session 1

1.1 Session 1 Copy 1

Collections

Problem Statement

You are tasked with merging the bill data of employees from two different months. Each employee has at most one bill per month. The goal is to merge the two sets of data and calculate the total bill amount for each employee. If an employee appears in both months, their bill amounts should be summed.

- **Class Employee** has the following members.
 - Private instance variables `String name` and `int billAmount`
 - Constructor to initialize the instance variables.
 - Method `addBillAmount(int additionalBill)` to add additional bill amounts.
 - Method `toString()` to display the employee's name and bill amount in the output format.
- **Class Test** has the following members:
 - Method `mergeEmployeeBills` accepts two maps (`billMap1` and `billMap2`) of employee data, where the key is the employee's ID and the value is an `Employee` object.
 - It merges the two maps into a single map (`totalBills`), combining the bill amounts of employees who appear in both maps.
 - The `main` method should:
 - * Accept user input for the employee data for two months .
 - * For each employee, the input contains the employee's ID, name, and bill amount.
 - * After accepting the data, the method should merge the two maps and print the total bill data for all employees.

What you have to do

- Implement the method `mergeEmployeeBills`

Test Cases

Public test case 1 (Input):

ID1 Krishna 1000

ID2 Ram 2000

ID3 Tony 3000

ID1 Krishna 1000
ID4 Kiara 1500
ID5 Beena 2000

Output:

ID1 -> Krishna:2000
ID2 -> Ram:2000
ID3 -> Tony:3000
ID4 -> Kiara:1500
ID5 -> Beena:2000

Test Cases

Public test case 2 (Input):

ID6 Arjun 2500
ID7 Riya 1800
ID8 Sanjay 1200
ID9 Priya 3000
ID10 Mohan 1500
ID6 Arjun 2500

Output:

ID6 -> Arjun:5000
ID7 -> Riya:1800
ID8 -> Sanjay:1200
ID9 -> Priya:3000
ID10 -> Mohan:1500

Private test case 1 (Input):

ID11 Sita 2200
ID12 Rohit 2700
ID13 Anil 1800
ID11 Sita 2200
ID14 Ramesh 2000
ID15 Pooja 3000

Output:

ID11 -> Sita:4400
ID12 -> Rohit:2700
ID13 -> Anil:1800
ID14 -> Ramesh:2000
ID15 -> Pooja:3000

Private test case 2 (Input):

```
ID16 Sneha 1000
ID17 Raj 2100
ID18 Alok 2400
ID19 Meera 1700
ID20 Neha 2000
ID16 Sneha 1000
```

Output:

```
ID16 -> Sneha:2000
ID17 -> Raj:2100
ID18 -> Alok:2400
ID19 -> Meera:1700
ID20 -> Neha:2000
```

Template Code

```
import java.util.*;
class Employee {
    String name;
    int billAmount;
    public Employee(String name, int billAmount) {
        this.name = name;
        this.billAmount = billAmount;
    }
    public void addBillAmount(int additionalBill) {
        this.billAmount += additionalBill;
    }

    public String toString() {
        return name + ":" + billAmount;
    }
}

public class Test {

    // Method to merge two maps of employee bills
    public static Map<String, Employee> mergeEmployeeBills(Map<String, Employee> billMap) {
        //Complete the definition
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
    }
}
```

```

Map<String, Employee> billMarch = new LinkedHashMap<>();
Map<String, Employee> billApril = new LinkedHashMap<>();
for (int i = 1; i <= 3; i++) {
    String id = scanner.next();
    String name = scanner.next();
    int billAmount = Integer.parseInt(scanner.next());

    billMarch.put(id, new Employee(name, billAmount));
}
for (int i = 1; i <= 3; i++) {
    String id = scanner.next();
    String name = scanner.next();
    int billAmount = Integer.parseInt(scanner.next());

    billApril.put(id, new Employee(name, billAmount));
}
Map<String, Employee> totalBill = mergeEmployeeBills(billMarch, billApril);
for (Map.Entry<String, Employee> entry : totalBill.entrySet()) {
    System.out.println(entry.getKey() + " -> " + entry.getValue());
}
}
}

```

Solution:

```

import java.util.*;
class Employee {
    String name;
    int billAmount;
    public Employee(String name, int billAmount) {
        this.name = name;
        this.billAmount = billAmount;
    }
    public void addBillAmount(int additionalBill) {
        this.billAmount += additionalBill;
    }

    public String toString() {
        return name + ":" + billAmount;
    }
}

```

```

public class Test {

    // Method to merge two maps of employee bills
    public static Map<String, Employee> mergeEmployeeBills(Map<String, Employee> billMap1, Map<String, Employee> billMap2) {
        Map<String, Employee> totalBills = new LinkedHashMap<>();
        for (Map.Entry<String, Employee> entry : billMap1.entrySet()) {
            totalBills.put(entry.getKey(), entry.getValue());
        }
        for (Map.Entry<String, Employee> entry : billMap2.entrySet()) {
            if (totalBills.containsKey(entry.getKey())) {
                Employee existingEmployee = totalBills.get(entry.getKey());
                existingEmployee.addBillAmount(entry.getValue().billAmount);
            } else {
                totalBills.put(entry.getKey(), entry.getValue());
            }
        }

        return totalBills;
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        Map<String, Employee> billMarch = new LinkedHashMap<>();
        Map<String, Employee> billApril = new LinkedHashMap<>();
        for (int i = 1; i <= 3; i++) {
            String id = scanner.next();
            String name = scanner.next();
            int billAmount = Integer.parseInt(scanner.next());

            billMarch.put(id, new Employee(name, billAmount));
        }
        for (int i = 1; i <= 3; i++) {
            String id = scanner.next();
            String name = scanner.next();
            int billAmount = Integer.parseInt(scanner.next());

            billApril.put(id, new Employee(name, billAmount));
        }
        Map<String, Employee> totalBill = mergeEmployeeBills(billMarch, billApril);
        for (Map.Entry<String, Employee> entry : totalBill.entrySet()) {
            System.out.println(entry.getKey() + " -> " + entry.getValue());
        }
    }
}

```

```
}  
}
```

1.2 Innerclass

Problem Statement

Write a Java program that processes a set of rental car reservation requests in a car rental agency. It accepts a specific number of cars as input, and prints whether the request is successful or not, based on the number of cars available. The code uses the concept of inner classes and has the following components:

- Interface **Rentable** with an abstract method **rentCar()**.
- Class **RentalAgency** representing a car rental agency:
 - It has an instance variable **name** to store the agency name.
 - Constructor to initialize **name**.
 - An inner private class **CarRental** that implements **Rentable** and has/does the following.
 - * Instance variables **reqCars** and **availCars** to hold the number of requested cars and available cars, respectively.
 - * Constructor to initialize the instance variables.
 - * Implement the method **public String rentCar()** that checks whether **requested cars** are less than **available cars**. If it is less, then returns "Cannot rent", else it returns "Rented successfully. Available cars are <available cars>".
 - Define a method **reserveCar(int cars)** that takes the number of cars to be reserved as an argument and returns a **Rentable** object.
- Class **FClass** with the **main** method:
 - Creates an object of **RentalAgency** with the **name** "RideSafe Rentals", It invokes the method **reserveCar** method to obtain a **Rentable** object which invokes the method **rentCar()** on it.

What you have to do

- Define method **reserveCar** in class **RentalAgency**

Template Code

```
import java.util.*;
interface Rentable {
    public abstract String rentCar();
}
class RentalAgency {
    String name;
```

```

public RentalAgency(String n) {
    this.name = n;
}

private class CarRental implements Rentable {
    int reqCars;
    int availCars;

    public CarRental(int cars) {
        reqCars = cars;
        availCars = 10;
    }

    public String rentCar() {
        if (availCars < reqCars)
            return "Cannot rent";
        else {
            availCars = availCars - reqCars;
            return "Rented successfully. Available cars are " + availCars;
        }
    }
}

//***** Define the method reserveCar() here
}

public class FClass {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        RentalAgency a1 = new RentalAgency("SafeRide Rentals");
        Rentable r1 = a1.reserveCar(sc.nextInt());
        System.out.println(r1.rentCar());
    }
}

```

Solution:

```

import java.util.*;
interface Rentable {
    public abstract String rentCar();
}
class RentalAgency {
    String name;

```

```

    public RentalAgency(String n) {
        this.name = n;
    }

    private class CarRental implements Rentable {
        int reqCars;
        int availCars;

        public CarRental(int cars) {
            reqCars = cars;
            availCars = 10;
        }

        public String rentCar() {
            if (availCars < reqCars)
                return "Cannot rent";
            else {
                availCars = availCars - reqCars;
                return "Rented successfully. Available cars are " + availCars;
            }
        }
    }

    public Rentable reserveCar(int cars) {
        return new CarRental(cars);
    }
}

public class FClass {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        RentalAgency a1 = new RentalAgency("SafeRide Rentals");
        Rentable r1 = a1.reserveCar(sc.nextInt());
        System.out.println(r1.rentCar());
    }
}

```

Public test case 1:

Input:

6

Output:

Rented successfully. Available cars are 4

Public test case 2:

Input:

12

Output:

Cannot rent

Private test case 1:

Input:

10

Output:

Cannot rent